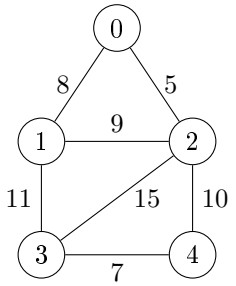
Ще изпълним алгоритъма на Дейкстра върху следния примерен граф:



Нека началният връх да бъде 0.

Основният цикъл ще направи 5 итерации. Стойностите в масивите d и $pred$ се изменят така:

$$d = (\underline{0},\ \infty,\ \infty,\ \infty,\ \infty), \qquad pred = (-,\ -,\ -,\ -,\ -)$$
$$d = (0,\ 8,\ \underline{5},\ \infty,\ \infty), \qquad pred = (-,\ 0,\ 0,\ -,\ -)$$
$$d = (0,\ \underline{8},\ 5,\ 20,\ 15), \qquad pred = (-,\ 0,\ 0,\ 2,\ 2)$$
$$d = (0,\ 8,\ 5,\ 19,\ \underline{15}), \qquad pred = (-,\ 0,\ 0,\ 1,\ 2)$$
$$d = (0,\ 8,\ 5,\ \underline{19},\ 15), \qquad pred = (-,\ 0,\ 0,\ 1,\ 2)$$

Подчертаните стойности показват кой връх се избира да влезе в множеството U на съответната итерация.

На първата итерация се избира началният връх 0 и масивът d се актуализира в неговите съседи $1, 2$.

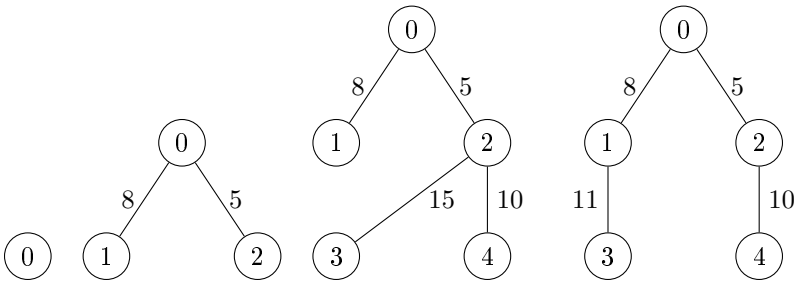
На втората итерация се избира върхът 2. Пътят до 1 през 2 има цена 14, така че $d[1]$ не се променя. Върховете 3 и 4 се достигат за първи път на тази итерация и пътищата им през 2 имат цена $d[3] = 20$ и $d[4] = 15$.

На третата итерация се избира върхът 1. Пътят до 3 през 1 има цена 19, така че $d[3]$ става 19 и съответно $pred[3]$ се променя на 1.

На четвъртата итерация се избира върхът 4. Пътят до 3 през 4 има цена 22, така че $d[3]$ не се променя.

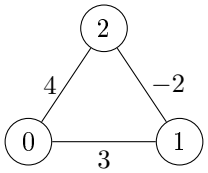
На последната пета итерация остава да се избере върхът 3. Разбира се, тук няма повече промени на стойности.

Дървото на текущите минимални пътища претърпява следните последователни промени:



За домашно изпълненете алгоритъма на Дейкстра и с другите възможности за начални върхове $1, 2, 3, 4$.

Да отбележим, че коректността на алгоритъма на Дейкстра зависи съществено от факта, че не допускаме ребра с отрицателни тегла. Тук имаме предвид, че търсим само прости минимални пътища (както отбелязохме по-горе, функцията *dist* не е добре дефинирана при наличие на ребра с отрицателно тегло, ако използваме произволни пътища). За да се убедим в това, да разгледаме следния пример:



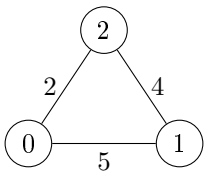
С начален връх 0 масивите d и $pred$ се изменят така:

$$d = (\underline{0},\ \infty,\ \infty), \qquad pred = (-,\ -,\ -)$$
$$d = (0,\ \underline{3},\ 4), \qquad pred = (-,\ 0,\ 0)$$
$$d = (0,\ 3,\ \underline{1}), \qquad pred = (-,\ 0,\ 1)$$

Тъй като търсим прости минимални пътища, промяна на стойността на d се допуска само за върховете извън U (и по тази причина стойността на $d[1]$ няма да се промени на -1 в последната итерация).

Както се вижда, резултатът е некоректен за минималния прост път от 0 до 1, който минава през върха 2 и има цена 2. Причината е, че алгоритъмът не може да избере по-скъпото ребро и игнорира възможността цената на пътя да може да стане по-малка от цената на това ребро.

Друг случай, в който не е приложим алгоритъмът на Дейкстра е при търсене на прости пътища с максимална цена. Да предположим, че обърнем неравенствата в алгоритъма и на всяка стъпка нека в U да влиза върхът с максимална стойност на d (разбира се, ще трябва да използваме $-\infty$ вместо ∞). Следният контрапример показва, че алгоритъмът няма да работи правилно:



Отново нека 0 е началният връх. Масивите d и $pred$ се изменят така:

$$\begin{aligned} d &= (\underline{0},\ -\infty,\ -\infty), & pred &= (-,\ -,\ -) \\ d &= (0,\ \underline{5},\ 2), & pred &= (-,\ 0,\ 0) \\ d &= (0,\ 5,\ \underline{9}), & pred &= (-,\ 0,\ 1) \end{aligned}$$

Максималният прост път от 0 до 1 има цена 6 и минава през 2.

Причината алгоритъмът на Дейкстра да не работи с отрицателни тегла и с максимални пътища е сходна в следния смисъл: задачата за намиране на прости максимални пътища в претеглен граф G е еквивалентна на задачата за намиране на прости минимални пътища в претегления граф $-G$, който се получава от G като се смени знака на цената на всяко ребро.

Оптимални покриващи дървета

Нека $G = (V, E)$ е свързан граф и $c : E \to \mathbb{R}$ е функция, задаваща цени (тегла) по ребрата.

За покриващо дърво $D = (V, E')$ на G дефинираме *цена (тегло) на дървото* D да бъде реалното число $c(D) = \sum_{e \in E'} c(e)$, т.е. сумата на цените на всички ребра, участващи в D.

Дефиниция 1 *Покриващото дърво* D_0 *наричаме* минимално, *ако* $c(D_0) \leq c(D)$ *за всяко покриващо дърво* D *на* G.
Покриващото дърво D_0 *наричаме* максимално, *ако* $c(D_0) \geq c(D)$ *за всяко покриващо дърво* D *на* G.

Лесно се съобразява, че такива дървета съществуват: графът G има краен брой покриващи дървета и във всяко крайно множество от реални числа има най-малко и най-голямо число.

За разлика от алгоритъма на Дейкстра, тук няма изискване цените на ребрата да са неотрицателни. Алгоритмите, които ще разгледаме намират минимално покриващо дърво. С тяхна помощ може да се намери и максимално покриващо дърво, ако първоначално се обърнат знаците на цените на всички ребра в графа.

Задача: Даден е свързан граф $G = (V, E)$ с тегла по ребрата $c : E \to \mathbb{R}$. Да се намери минимално покриващо дърво $D = (V, E')$ на графа G.

Първо ще разгледаме *алгоритъма на Прим*.

Идеята в алгоритъма е да присъединяваме върховете един след друг по такъв начин, че цената на присъединяващото ребро на всяка стъпка да бъде възможно най-малка. Имаме свобода да започнем с произволен връх.

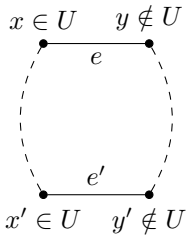
Инициализация: избираме $r \in V$, $U = \{r\}$; $E' = \emptyset$;
Основен цикъл на алгоритъма:
докато $U \neq V$,
нека $e = \{x, y\}$ е ребро, за което $x \in U$, $y \notin U$ и $c(\{x, y\})$ е минимална;
присъединяваме y към x в U: $U = U \cup \{y\}$; $E' = E' \cup \{e\}$;

Теорема 2 (коректност на алгоритъма на Прим) *След изпълнението на алгоритъма, $D = (V, E')$ е минимално покриващо дърво на графа G.*

Доказателство. Ще докажем, че на всяка итерация на основния цикъл съществува минимално покриващо дърво, което разширява дървото (U, E'). Разбира се, това ще е изпълнено и в края на алгоритъма: има минимално покриващо дърво, разширяващо дървото (V, E'), но това може да се случи само ако двете дървета съвпадат (върховете им са едни и същи).

База: В началото $(U, E') = (\{r\}, \emptyset)$, така че произволно минимално покриващо дърво върши работа (поне едно съществува, както отбелязохме).

Стъпка: Нека $D_0 = (V, E_0)$ е минимално покриващо дърво, което разширява (U, E') ($E' \subseteq E_0$) на някоя итерация (това е индукционното предположение). Нека $e = \{x, y\}$ е реброто, което е избрано на тази итерация. Лесният случай е, когато $e \in E_0$, т.е. когато реброто e участва в минималното покриващо дърво D_0. Тогава D_0 разширява $(U \cup \{y\}, E' \cup \{e\})$ и сме готови. Да разгледаме другия случай: $e \notin E_0$. Добавяме реброто e към минималното покриващо дърво D_0. Знаем, че тогава се получава единствен цикъл, в който участва e. Цикълът започва с $x \in U$ и стига до $y \notin U$, преди да се завърне в x. За да стане това, със сигурност има друго ребро $e' = \{x', y'\}$, такова че $x' \in U$, $y' \notin U$:



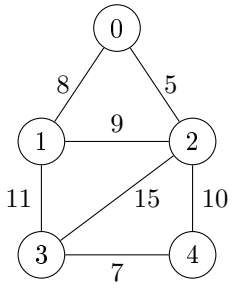
За реброто e' имаме $e' \in E_0$, $e' \notin E'$. От избора на e в алгоритъма, имаме неравенството $c(e) \leq c(e')$. Да образуваме $D_0' = (V, E_0 \setminus \{e'\} \cup \{e\})$. Ясно е, че D_0' също е покриващо дърво на G. За неговата цена имаме:

$$c(D_0') \;=\; c(D_0) - c(e') + c(e) \;\leq\; c(D_0).$$

Тъй като дървото D_0 е минимално, D_0' също е минимално (всъщност дори имаме равенство и $c(e) = c(e')$). Тъй като $E' \cup \{e\} \subseteq E_0 \setminus \{e'\} \cup \{e\}$, дървото D_0' разширява $(U \cup \{y\}, E' \cup \{e\})$, така че доказателството е завършено.

Идеята на доказателството може да се обобщи по следния начин: в началото сме с "празни" ръце и минималното дърво, което търсим е "на светлинни години" от нас. Това което правим е следното: избираме едно подходящо ребро, с идеята да "приближим" решението към нас. Оказва се, че това подходящо ребро или е в решението, което търсим или не е, но тогава то "магически" се трансформира в друго решение, което съдържа избраното ребро. Така стъпка по стъпка се приближаваме все повече до решението, като го заставяме да се променя и накрая успяваме да го "хванем" изцяло.

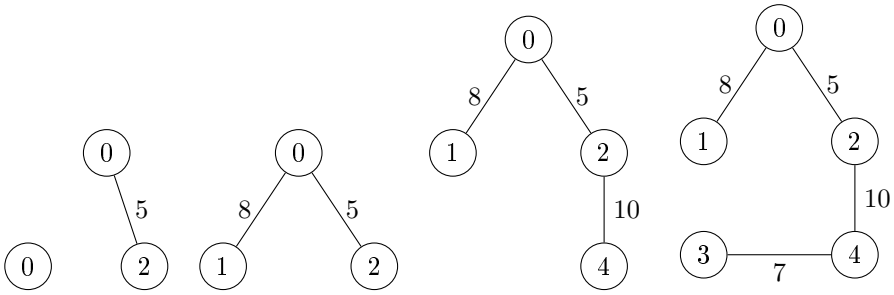
Да изпълним алгоритъма на Прим върху същия примерен граф от по-горе:



Нека началният връх е 0. Множеството U се променя по следния начин:

$$U = \{0\},\ \ U = \{0, 2\}\ (e = \{0, 2\}),\ \ U = \{0, 1, 2\}\ (e = \{0, 1\}),$$
$$U = \{0, 1, 2, 4\}\ (e = \{2, 4\}),\ \ U = \{0, 1, 2, 3, 4\}\ (e = \{4, 3\}),$$

като постепенно се получава минималното покриващо дърво:



Може да проверите, че за примерния граф никое от дърветата, получени с алгоритъма на Дейкстра не съвпада с минималното покриващо дърво, получено с алгоритъма на Прим. Въпреки, че има сходство в двата алгоритъма, при Дейкстра разширението става по "локален" критерий (отдалеченост от фиксираното начало v_0), докато при Прим разширението е по "глобален" критерий (отдалеченост от всички добавени до момента върхове).

По-нататък ще разгледаме *алгоритъма на Крускал*.

Отново ще добавяме постепенно ребра, докато получим търсеното минимално покриващо дърво. В началото сортираме ребрата по нарастваща стойност на тяхната цена. След това опитваме да ги добавим едно след друго, като внимаваме да не се получи цикъл (тогава просто игнорираме съответното ребро).

По-конкретно, ще поддържаме множество от дървета X. Множествата от върховете на тези дървета през целия алгоритъм ще образуват разбиване на V. В края на алгоритъма X ще бъде синглетон, като единственият му елемент ще е търсеното минимално покриващо дърво. За всеки връх $v \in V$, нека $T_v = (V_v, E_v)$ е текущото дърво от X, такова че $v \in V_v$.

Инициализация: сортираме ребрата по нарастваща цена: $e_1, e_2, \ldots, e_m$, където $c(e_1) \leq c(e_2) \leq \ldots \leq c(e_m)$. За всеки връх $v \in V$ образуваме тривиално дърво $T_v = (\{v\}, \emptyset)$ и нека $X = \{T_v \mid v \in V\}$.

Основен цикъл на алгоритъма:

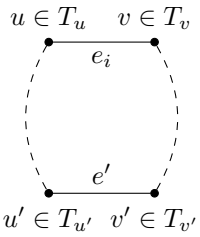
за $i = 1, 2, \ldots, m$
  нека $e_i = \{u, v\}$ и T_u, T_v са съответните дървета от X
  ако $T_u \neq T_v$, то
    образуваме $T = (V_u \cup V_v,\ E_u \cup E_v \cup \{e_i\})$;
    променяме $X = X \setminus \{T_u, T_v\} \cup \{T\}$;

Теорема 3 (коректност на алгоритъма на Крускал) *След изпълнението на алгоритъма, X съдържа единствен елемент, който е минимално покриващо дърво на графа G.*

Доказателство. Ще приложим същата идея, както при доказателството на коректността на алгоритъма на Прим. Първо ще докажем, че на всяка итерация на основния цикъл, съществува минимално покриващо дърво, което разширява обединението (V, E') на дърветата в X.

База: В началото $E' = \emptyset$, така че произволно минимално покриващо дърво върши работа.

Стъпка: Нека $D_0 = (V_0, E_0)$ е минимално покриващо дърво, което разширява обединението (V, E') на X на някоя итерация (индукционно предположение). На следващата итерация нека преглеждаме реброто $e_i = \{u, v\}$. В случай, че $T_u = T_v$, множеството X не се променя, така че същото минимално покриващо дърво D_0 върши работа. Нека сега $T_u \neq T_v$. Тогава според алгоритъма добавяме e_i към E'. Ако се окаже, че $e_i \in D_0$, то дървото D_0 разширява и новото обединение $(V, E' \cup \{e_i\})$, така че сме готови. Нека $e_i \notin E_0$. Както в доказателството на алгоритъма на Прим, добавяме e_i към E_0 и така се образува единствен цикъл. Тъй като $T_u \cap T_v = \emptyset$, в този цикъл трябва да има друго ребро $e' = \{u', v'\}$, което свързва две различни дървета $T_{u'}$ и $T_{v'}$ от X:



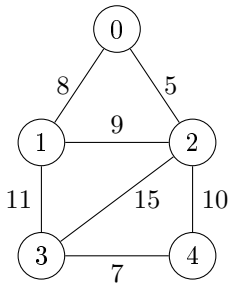
За реброто e' имаме: $e' \in E_0$ и $e' \notin E'$, тъй като всички ребра в E' свързват върхове от едно и също дърво от X. Но тогава не е възможно да имаме $c(e') < c(e_i)$, тъй като това означава, че реброто e' е разгледано на предходна итерация и не е добавено, въпреки че свързва различни дървета и на тази предходна итерация. Така $c(e_i) \leq c(e')$. По-нататък разсъждението повтаря това в алгоритъма на Прим. Образуваме $D'_0 = (V, E_0 \setminus \{e'\} \cup \{e_i\})$. Това е покриващо дърво, за което

$$c(D'_0) \;=\; c(D_0) - c(e') + c(e_i) \;\leq\; c(D_0),$$

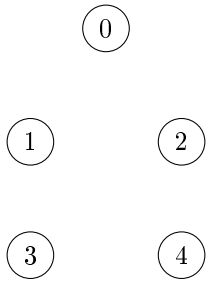
така че D'_0 е минимално, тъй като D_0 е минимално. Освен това, D'_0 разширява $(V, E' \cup \{e_i\})$, така че доказателството на тази част е завършено.

Да разгледаме окончателното множество X, което е образувано след края на изпълнението на основния цикъл. От това което доказахме знаем, че можем да изберем минимално покриващо дърво $D_0 = (V, E_0)$, което разширява обединението (V, E') на дърветата в X. Да допуснем, че има ребро $e_i \in E_0$, такова че $e_i \notin E'$. Тогава на итерация i, реброто $e_i = \{u, v\}$ не е добавено, т.е. $T_u = T_v$. Тъй като u и v са в едно и също дърво, между тях съществува път в това дърво и той се състои от ребра в E'. Тъй като $E' \subseteq E_0$ и $e_i \in E_0$ ще излезе, че в D_0 има цикъл - противоречие. Получихме, че $E' = E_0$. Тъй като D_0 е свързан граф, това е възможно само ако X съдържа единствен елемент D_0.

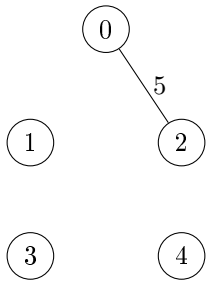
Да изпълним алгоритъма на Крускал върху примерния граф:



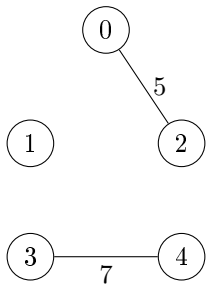
В началото имаме тривиални дървета от всеки връх:



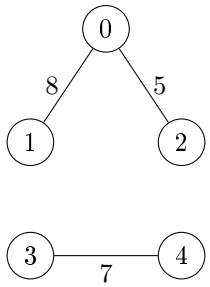
На първата итерация разглеждаме $e_1 = \{0, 2\}$ с цена 5. Тъй като 0 и 2 са в различни дървета ги обединяваме:



На втората итерация $e_2 = \{3, 4\}$ с цена 7. Върховете 3 и 4 са в различни дървета, затова добавяме e_2:

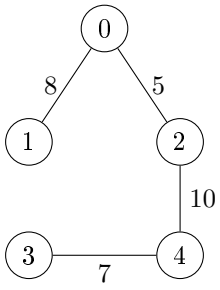


На третата итерация $e_3 = \{0, 1\}$ с цена 8 и го добавяме:



На четвъртата итерация $e_4 = \{1, 2\}$ с цена 9. Върховете 1 и 2 са в едно и също дърво, така че игнорираме e_4.

На петата итерация $e_5 = \{2, 4\}$ с цена 10. Добавяме го и така получаваме:



Разбира се, можем да приключим на тази итерация, тъй като всички останали ребра ще бъдат игнорирани.